

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.Б71-мм

Консольный архиватор файлов на основе алгоритма Хаффмана

КАЗАЧКОВ Роман Андреевич

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
старший преподаватель, к. т. н., Литвинов Ю. В.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Кодирование Хаффмана	5
2.2. Существующие решения	5
2.3. Выводы	6
3. Описание решения	7
3.1. Формат архива	7
3.2. Архитектура реализации	7
3.3. Сборка и непрерывная интеграция	8
4. Эксперимент	9
4.1. Условия и исследовательские вопросы	9
4.2. Проверка корректности	9
4.3. Метрики и результаты измерений	10
4.4. Ограничения воспроизводимости	11
Заключение	12
Список литературы	13

Введение

Сжатие без потерь требуется в задачах, где восстановленный результат должен в точности совпадать с исходными данными: при хранении текстов, программ, документов и произвольных бинарных файлов. Одним из базовых методов энтропийного кодирования является алгоритм Хаффмана, предложенный Д. А. Хаффманом в 1952 г. [4]. Он сопоставляет частым символам короткие двоичные слова, а редким — более длинные, сохраняя возможность однозначного декодирования.

Промышленные форматы архивов, такие как `gzip` и `ZIP`, применяют кодирование Хаффмана не изолированно, а в составе алгоритма `DEFLATE` вместе с поиском повторяющихся последовательностей [1, 3]. Поэтому на их примере непосредственное влияние дерева Хаффмана на размер и время обработки данных наблюдать неудобно. Для изучения алгоритма полезен самостоятельный инструмент с простым документированным форматом файла, воспроизводимой сборкой и автоматическими проверками.

Целью работы является разработка на языке `C` консольного архиватора, который сжимает и без потерь восстанавливает текстовые и бинарные файлы при помощи статического кодирования Хаффмана, а также проверка его корректности и исследование поведения на данных разных типов.

В работе предложен архивный формат, содержащий размер исходного файла и таблицу частот всех байтов; реализованы модули построения дерева, побитового ввода-вывода и кодирования; подготовлены тесты и скрипты измерений. Работа выполнена в рамках учебной практики второго семестра. Исходный код открыт и доступен в репозитории проекта¹.

¹Репозиторий «Huffman Archiver», URL: <https://github.com/RomanKazz/huffman-archiver> (дата обращения: 27 мая 2026 г.).

1. Постановка задачи

Конечным пользователем решения является пользователь командной строки, которому необходим небольшой архиватор и воспроизводимая реализация алгоритма Хаффмана для учебных экспериментов. Пользователь должен иметь возможность сжать произвольный файл, восстановить его без потерь и повторить измерения на опубликованном коде.

Целью работы является реализация архиватора файлов на основе статического кодирования Хаффмана и экспериментальная проверка его свойств. Для достижения цели были поставлены следующие задачи:

1. выполнить обзор алгоритма Хаффмана и его применения в существующих средствах сжатия;
2. спроектировать формат архива и реализовать сжатие и распаковку текстовых и бинарных файлов;
3. обеспечить проверку корректности реализации модульными и интеграционными тестами, а также средствами автоматического анализа;
4. провести измерения коэффициента сжатия и времени обработки на данных с различными свойствами и проанализировать результаты.

Результатом работы должен быть опубликованный репозиторий консольной утилиты с инструкцией по сборке и запуску, описанием формата архива, автоматическими проверками и воспроизводимым сценарием измерений.

2. Обзор

2.1. Кодирование Хаффмана

Алгоритм Хаффмана строит оптимальный префиксный код для известного распределения частот символов [4]. Сначала для каждого символа создаётся лист дерева с весом, равным его частоте. Затем два узла с минимальными весами объединяются во внутренний узел с весом, равным сумме весов объединённых узлов; операция повторяется, пока не останется один корень. Путь от корня к листу задаёт двоичный код символа: переход влево соответствует биту 0, переход вправо — биту 1.

Префиксность означает, что ни один код символа не является началом другого кода. Поэтому декодирование выполняется последовательным обходом дерева по битам входного потока: достижение листа однозначно определяет восстановленный символ. В статическом варианте кодирования распределение частот определяется до начала записи сжатых данных; декодеру необходимо получить сведения, позволяющие восстановить то же дерево.

2.2. Существующие решения

В алгоритме DEFLATE кодирование Хаффмана применяется после этапа LZ77, который заменяет повторяющиеся фрагменты ссылками на уже обработанные данные [1]. Такой подход используется библиотекой zlib и утилитой gzip [3, 2]. Его преимуществом является более эффективное сжатие типичных файлов, поскольку учитываются не только частоты отдельных значений, но и повторяющиеся последовательности. Недостаток для целей данной работы состоит в том, что результат измерений характеризует комбинацию алгоритмов, а не отдельное кодирование Хаффмана.

Статический архиватор, реализованный в работе, занимает другую позицию: он хранит таблицу частот и применяет только побайтовое кодирование Хаффмана. Он уступает универсальным промышленным архиваторам как средство достижения максимального коэффициента

сжатия, но позволяет непосредственно изучать зависимость результата от распределения байтов и сохраняет простой формат архива.

Кроме промышленных архиваторов, существуют учебные и справочные реализации кодирования Хаффмана. Например, Rosetta Code содержит набор реализаций алгоритма на разных языках программирования [6], а проект Nayuki Reference Huffman coding предоставляет компактную справочную реализацию кодирования и декодирования [5]. Такие материалы полезны как иллюстрация алгоритма, но обычно не ставят акцент на переносимом бинарном формате архива, обработке граничных случаев и воспроизводимом эксперименте. В данной работе эти аспекты выделены как самостоятельные инженерные требования.

2.3. Выводы

Для реализации выбран статический вариант алгоритма Хаффмана. Он соответствует цели работы, поскольку обеспечивает декодирование произвольных бинарных данных, допускает явное описание формата архива и позволяет измерить свойства именно энтропийного кодирования без влияния словарных методов.

3. Описание решения

3.1. Формат архива

Разработанный архив начинается с заголовка фиксированного размера 2060 байт, после которого располагается сжатый битовый поток. Все размеры в формате хранятся в байтах, если не указано обратное. Заголовок позволяет проверить тип файла, восстановить дерево и прекратить декодирование после получения исходного количества байтов. Формат приведён в таблице 1.

Таблица 1: Формат создаваемого архива.

Поле	Размер	Назначение
<code>magic</code>	4 байта	сигнатура HUFF
<code>size</code>	8 байт	размер исходного файла в байтах, <code>uint64_t</code> , little-endian
<code>freq[256]</code>	2048 байт	таблица частот возможных байтов
<code>data</code>	переменный	битовый поток кодов Хаффмана

Таблица частот увеличивает размер малых архивов, но делает формат самодостаточным: распаковка не требует отдельного словаря или внешних метаданных. Представление размера файла и частот типом `uint64_t` исключает ограничение формата размером в 4 ГиБ. Файлы размером не менее 2^{64} байт данным форматом не поддерживаются, поскольку их размер нельзя представить в поле `size`.

3.2. Архитектура реализации

Реализация написана на языке C стандарта C11 и разделена на следующие модули:

- `main.c` разбирает аргументы командной строки и выбирает режим сжатия или распаковки;
- `huffman_codec.c` подсчитывает частоты, формирует заголовок, строит таблицу кодов и выполняет декодирование;

- `huffman_tree.c` строит и освобождает дерево Хаффмана, используя очередь с приоритетом на основе двоичной кучи;
- `bitio.c` реализует буферизованную запись и чтение отдельных битов в файловом потоке.

При сжатии файл читается сначала для подсчёта частот, затем повторно для записи кодовых слов в битовый поток. При распаковке читается заголовок, по таблице частот восстанавливается дерево и из потока выводится ровно число байтов, заданное полем `size`. Специально обрабатаны пустой вход и файл, состоящий из единственного различного символа: во втором случае символу назначается однокбитовый код.

3.3. Сборка и непрерывная интеграция

Сборка проекта описана средствами `CMake`; при стандартной конфигурации для исполняемого файла и тестов включаются `AddressSanitizer` и `UndefinedBehaviorSanitizer`. В репозитории настроены процессы `GitHub Actions`: сборка, модульные и интеграционные тесты, контроль форматирования `clang-format`, проверка под `Valgrind`, а также статический анализ `CodeQL`. В `README` приведены команды сборки и использования, описание формата архива и инструкция по запуску экспериментов. Исходный код распространяется под лицензией `MIT`.

4. Эксперимент

4.1. Условия и исследовательские вопросы

Проверка результата воспроизводилась 27 мая 2026 г. на компьютере с macOS (Darwin 25.4.0), AppleClang 21.0.0 и CMake 4.3.2. Проект компилировался стандартной конфигурацией CMake с флагами `-O2`, `-fsanitize=address,undefined` и `-fno-omit-frame-pointer`, то есть с включёнными `AddressSanitizer` и `UndefinedBehaviorSanitizer`; в непрерывной интеграции дополнительно предусмотрена проверка тестов под `Valgrind`. Для проверки были сформулированы следующие вопросы:

RQ1: восстанавливает ли утилита исходные данные без потерь для типовых и граничных входов?

RQ2: как распределение байтов во входном файле влияет на коэффициент сжатия статическим кодированием Хаффмана?

RQ3: какое время требуется на сжатие и распаковку файлов выбранного размера в подготовленном сценарии измерений?

4.2. Проверка корректности

Модульные тесты проверяют построение дерева, чтение и запись заголовка, побитовый ввод-вывод, основной кодек, случайные данные и обработку повреждённого входа. Интеграционный сценарий последовательно сжимает и распаковывает тестовые файлы, а затем сравнивает восстановленные данные с исходными. Итоги запуска приведены в таблице 2.

Интеграционные входы включают пустой файл, файл из повторяющегося символа, текст, набор всех значений байта и случайные бинарные данные. Таким образом, на вопрос RQ1 получен положительный ответ для предусмотренного набора проверок: все запущенные тесты прошли успешно.

Таблица 2: Результаты проверки корректности реализации.

Проверка	Проверяемые свойства	Результат
Модульные тесты <code>ctest</code>	дерево, заголовок, битовый ввод-вывод, кодек, случайные данные, повреждённый вход	6/6
Интеграционный сценарий <code>tests/test.sh</code>	полный цикл сжатие–распаковка и побайтовое сравнение файлов	6/6

4.3. Метрики и результаты измерений

Сценарий `experiments/run_all.sh` генерирует текстовые, повторяющиеся, случайные и бинарно-подобные файлы размером 1, 10, 100 КиБ и 1 МиБ. Бинарно-подобный набор строится повторением системного исполняемого файла, поэтому имитирует не случайный поток байтов, а фрагменты реального бинарного файла. Кроме синтетических входов, сценарий добавляет типовые файлы: объединённый файл исходного кода на C, PDF-файл отчёта и системный исполняемый файл. Для каждого файла выполняется 10 запусков сжатия и распаковки. Основной метрикой размера является отношение размера архива к размеру исходного файла; значение меньше единицы означает уменьшение файла. Для времени приводятся среднее значение и выборочное среднеквадратичное отклонение. В таблице 3 приведены результаты для представительных входов.

Таблица 3: Обработка представительных входов, 10 запусков; время указано в миллисекундах (меньше — лучше).

Вход	Отношение размеров	Сжатие, мс	Распаковка, мс
Повторяющийся символ, 1 МиБ	0,1270	$115,0 \pm 2,7$	$83,7 \pm 1,6$
Текст, 1 МиБ	0,5064	$138,6 \pm 1,4$	$107,6 \pm 2,3$
Случайные байты, 1 МиБ	1,0020	$150,8 \pm 2,6$	$135,0 \pm 1,3$
Исходный код C, 16 КиБ	0,7744	$64,7 \pm 1,0$	$64,8 \pm 1,0$
PDF-отчёт, 65 КиБ	1,0302	$69,1 \pm 0,7$	$68,6 \pm 1,1$
Системный бинарный файл, 151 КиБ	0,4205	$74,3 \pm 1,2$	$69,8 \pm 0,8$

По вопросу RQ2 эксперимент показывает ожидаемую зависимость:

файл с одним доминирующим символом уменьшается приблизительно до 12,7% исходного размера, текстовый файл — до 50,6%, а равномерно случайные байты практически не сжимаются. Для малых файлов накладные расходы заголовка становятся определяющими: архив случайного входа размером 1 КиБ имеет отношение размеров 2,9922. На типовых файлах результат различается: исходный код C сжимается до 77,4% исходного размера, PDF-файл немного увеличивается из-за собственного сжатия внутри формата, а системный бинарный файл уменьшается до 42,1%. По вопросу RQ3 для файлов размером 1 МиБ время сжатия в измеренном наборе составляет от $115,0 \pm 2,7$ до $150,8 \pm 2,6$ мс, время распаковки — от $83,7 \pm 1,6$ до $135,0 \pm 1,3$ мс.

4.4. Ограничения воспроизводимости

Сценарии эксперимента позволяют повторить измерения, но результаты не являются бит-в-бит детерминированными. Файлы из набора `random` создаются из `/dev/urandom`, поэтому при каждом запуске имеют другое содержимое. Наборы `binary` и `real/system_binary` строятся на основе системного исполняемого файла `/bin/ls` или значения переменной окружения `BINARY_SOURCE`; следовательно, их содержимое зависит от операционной системы и выбранного исходного файла, хотя размер синтетического бинарного входа фиксируется скриптом.

Временные измерения зависят от процессора, нагрузки системы, версии компилятора и режима сборки. Поэтому приведённые значения следует рассматривать как характеристику конкретного окружения запуска, а не как переносимую оценку производительности на любой платформе. Для повторения полного сценария также требуются утилиты `python3` и `bc`. Пакеты `Python pandas` и `matplotlib` нужны только для построения графиков: при их отсутствии сценарий выводит таблицу агрегатов и пропускает генерацию изображений. Дополнительным развитием эксперимента является проверка на более широком наборе пользовательских файлов, например видеофайлов и документов разных форматов.

Заключение

В рамках учебной практики второго семестра получены следующие результаты:

- выполнен обзор статического кодирования Хаффмана и его применения в составе существующих алгоритмов сжатия;
- разработан консольный архиватор на языке C с документированным бинарным форматом и поддержкой текстовых и бинарных файлов;
- корректность реализации проверена модульными и интеграционными тестами: в воспроизведённом запуске успешно прошли 6 из 6 модульных наборов тестов и обработаны 6 из 6 интеграционных входных файлов;
- проведены измерения и приведены результаты для синтетических и типовых файлов; подтверждено эффективное сжатие избыточных данных, отсутствие выигрыша для равномерно случайного входа и зависимость результата от внутренней структуры реального файла.

Исходный код, документация по сборке, описание формата и сценарии экспериментов опубликованы в репозитории проекта: <https://github.com/RomanKazz/huffman-archiver>.

Для проекта настроены автоматические проверки сборки, тестов, форматирования и статического анализа при изменениях в репозитории. Дальнейшим развитием работы может быть фиксация входных данных эксперимента для полностью детерминированного сравнения, сопоставление результата с форматами, использующими DEFLATE, и расширение набора измерений более крупными пользовательскими файлами разных форматов.

Список литературы

- [1] DEFLATE Compressed Data Format Specification version 1.3 : Rep. : RFC 1951 / Internet Engineering Task Force ; Executor: L. Peter Deutsch : 1996. — URL: <https://www.rfc-editor.org/rfc/rfc1951> (дата обращения: 27 мая 2026 г.).
- [2] Free Software Foundation. GNU Gzip. — URL: <https://www.gnu.org/software/gzip/> (дата обращения: 27 мая 2026 г.).
- [3] Gailly Jean-loup, Adler Mark. zlib: A Massively Spiffy Yet Delicately Unobtrusive Compression Library. — URL: <https://zlib.net/> (дата обращения: 27 мая 2026 г.).
- [4] Huffman David A. A Method for the Construction of Minimum-Redundancy Codes // [Proceedings of the IRE](#). — 1952. — Vol. 40, no. 9. — P. 1098–1101. — URL: <https://ieeexplore.ieee.org/document/4051119> (дата обращения: 27 мая 2026 г.).
- [5] Nayuki. Reference Huffman coding. — URL: <https://www.nayuki.io/page/reference-huffman-coding> (дата обращения: 10 июня 2026 г.).
- [6] Rosetta Code contributors. Huffman coding. — URL: https://rosettacode.org/wiki/Huffman_coding (дата обращения: 10 июня 2026 г.).